1.) Understand and implement the different ER diagram notation with their relationship and Cardinalities.

Objective: -To understand different ER diagram notations and the relationships and cardinalities used in database modeling.

An Entity Relationship (ER) diagram is a graphical tool used to design and represent the structure of a database. It helps in visualizing entities, their attributes, and how they are connected with each other. ER diagrams are mainly used during the planning phase of database development.

- **Basic ER Diagram Notations**

i.) Entity

- Represented by a rectangle
- An entity is a real-world object or concept
- Example: Student, Employee, Department



Entity

ii.) Attribute

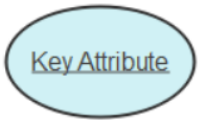
- Represented by an oval
- Describes properties of an entity
- Example: Name, Age, Salary



Attribute

iii.) **Primary Key Attribute**

- Underlined oval
- Uniquely identifies each record
- Example: Roll_No, Emp_ID



Key Attribute

iv.) Relationship

- Represented by a diamond
- Shows connection between entities

- Example: Works_in, Manages



Relationship

v.) **Weak Entity: -**

- Represented by **double rectangle**
- Depends on another entity for identification



Weak Entity

vi.) **Multivalued Attribute: -**

- Represented by **double oval**
- Example: Phone numbers



vii.) **Derived Attribute: -**

- Represented by **dotted oval**
- Example: Age derived from Date of Birth



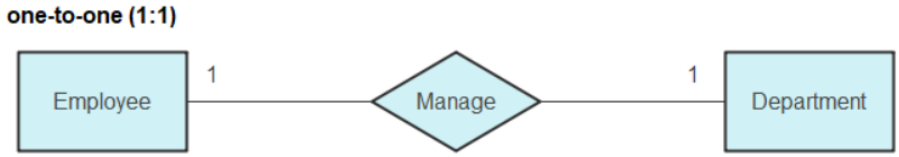
Derived Attribute

- **Relationships in ER Diagram**

  A relationship defines how two or more entities are associated.

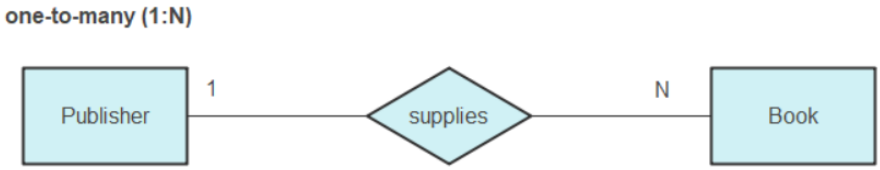
➢ **Types of relationships:**

1. **One-to-One (1:1) : -**
   - One entity is related to only one entity
   - Example: Employee → Department



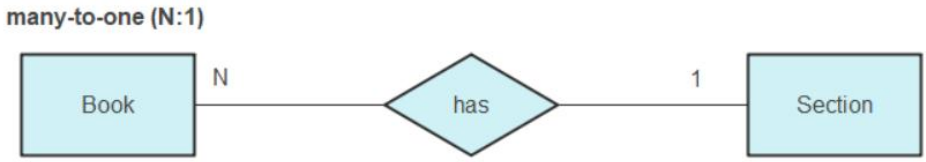
2. **One-to-Many (1:N): -**

   - One entity relates to many entities
   - Example: Publisher → Book



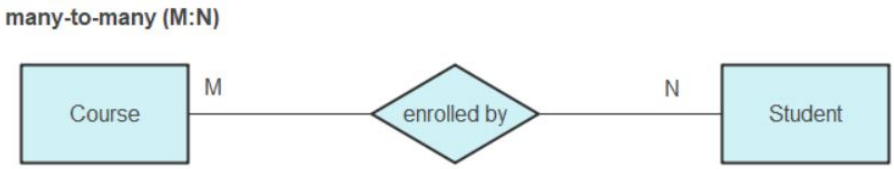
3. **Many-to-One (N:1): -**

   - A many-to-one relationship means many records of one entity are related to only one record of another entity.
   - Example: Book →Section



4. **Many-to-Many (M:N): -**

   - Many entities relate to many entities
   - Example: Students → Courses



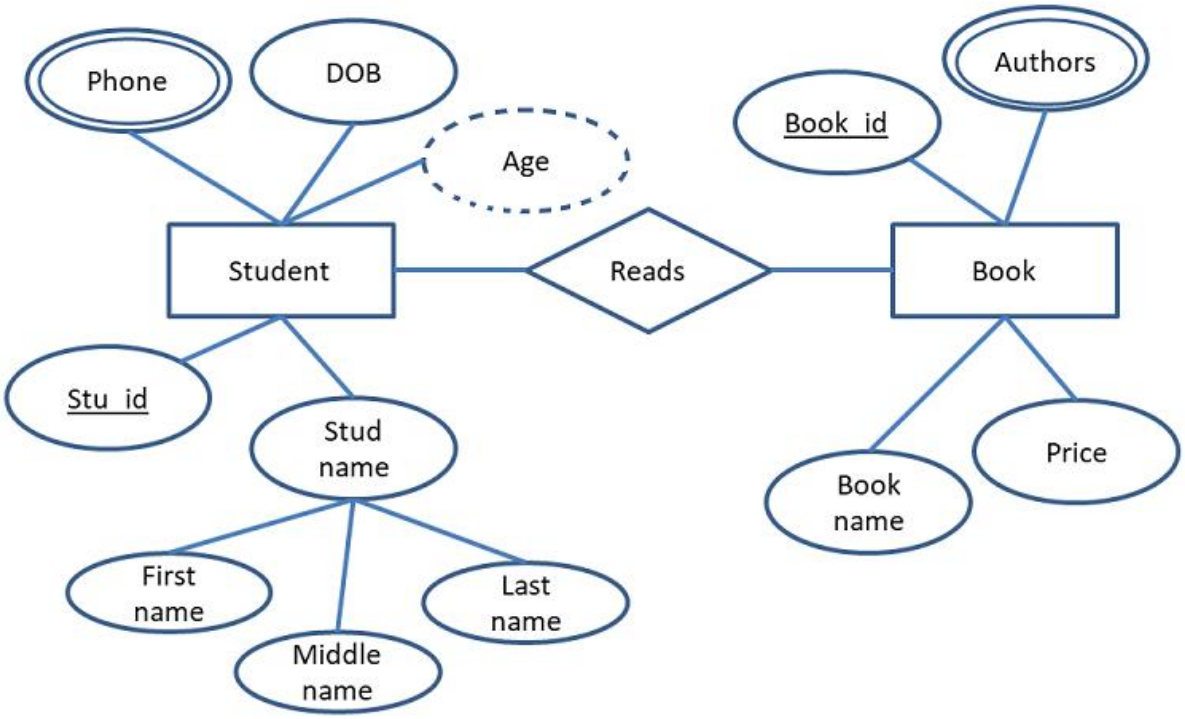
- **Cardinality: -**

  Cardinality specifies the number of entity instances that can participate in a relationship.

  Examples:

  - A department can have many employees (1:N)
  - An employee works in only one department
  - A student can enroll in many courses (M:N)

- **Example of ER Model: -**

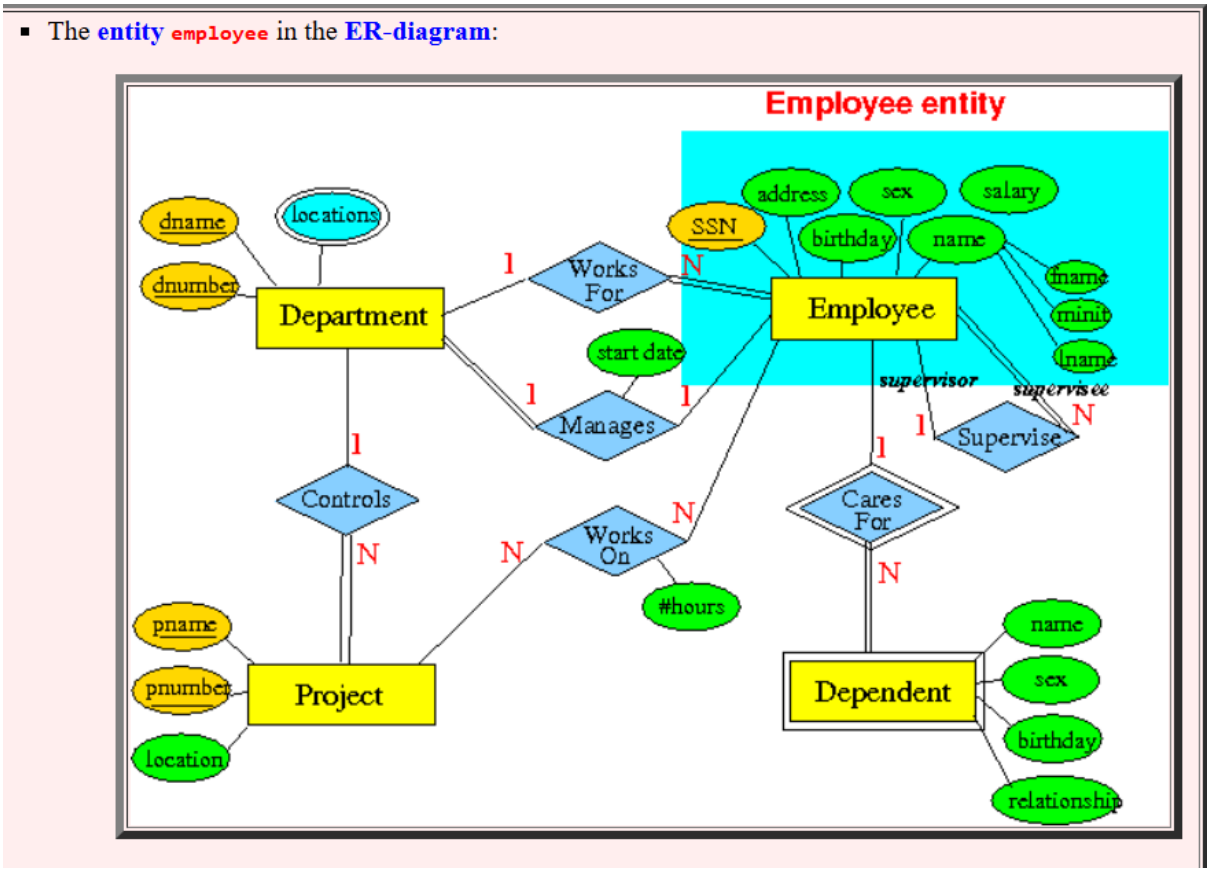


2.) Creating ER Diagram for company Database. Company database have entities like employee, departments, projects and dependents also implement the relationship and cardinalities between the entities with their relevant attribute.

Objective: - To design an ER diagram for a company database including entities Employee, Department, Project and Dependent, and implement relationships with proper cardinalities and attributes.

A company database stores information about employees, departments, projects, and dependents. ER modeling helps organize this information by defining entities, attributes, and relationships clearly before creating the actual database.

- **ER Diagram for Company Database: -**



❖ **Procedure: -**

1. Identify entities in the company database.
2. Assign attributes to each entity.
3. Define relationships between entities.
4. Assign proper cardinalities.
5. Draw the ER diagram using standard notation.

❖ **Entities and Attributes: -**

1. **Employee**
    - Emp_ID (Primary Key)
    - Name
    - Address
    - Salary
    - Gender
    - Dept_ID (Foreign Key)
2. **Department**
    - Dept_ID (Primary Key)
    - Dept_Name
    - Location
3. **Project**
    - Proj_ID (Primary Key)
    - Proj_Name
    - Budget
4. **Dependent**
    - Dep_ID (Primary Key)
    - Dep_Name
    - Relationship
    - Emp_ID (Foreign Key)

❖ **Relationships and Cardinalities: -**

1. **Department – Employee**
    - Relationship: *Works In*
    - Cardinality: **1 : N**
    - One department has many employees
    - Each employee belongs to one department
2. **Employee – Project**
    - Relationship: *Works On*
    - Cardinality: **M : N**
    - Many employees work on many projects

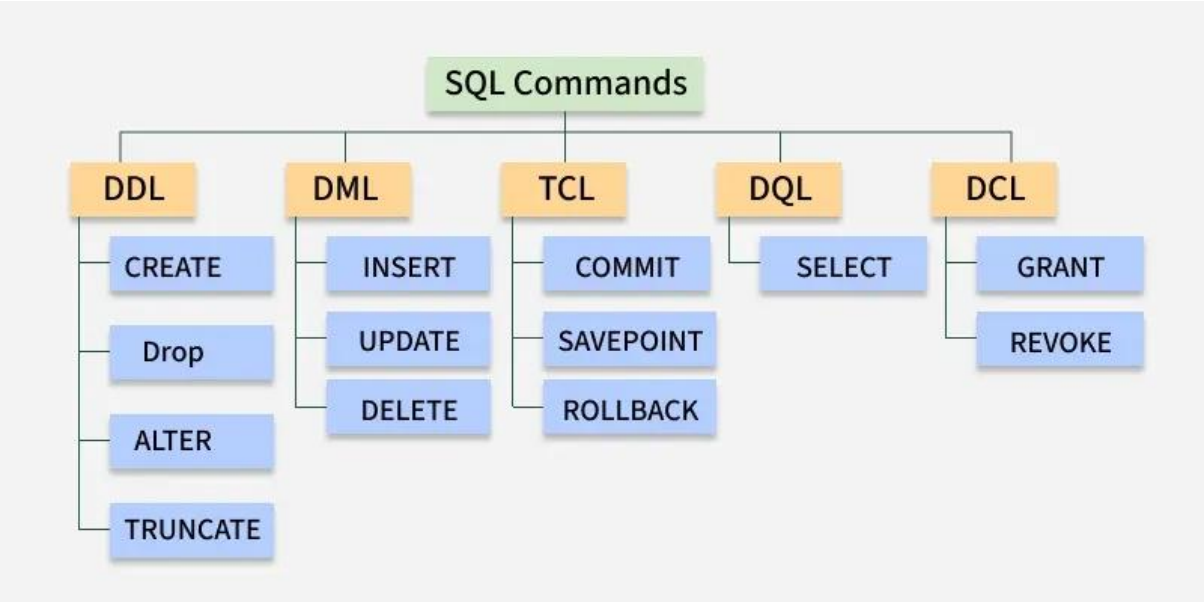
3. **Employee – Dependent**
    - Relationship: *Has*
    - Cardinality: **1 : N**
    - One employee can have many dependents
    - Each dependent belongs to one employee

- **Result: -**

  The ER diagram for the company database was created successfully with correct entities, attributes, relationships, and cardinalities.

3 Implement DDL, DML, DCL & TCL commands.

Objective: - To study and implement different SQL commands including DDL, DML, DCL and TCL.



i.) DDL Commands (Data Definition Language): -

- DDL commands are used to create and modify database structure.
- DDL (Data Definition Language) consists of SQL commands that can be used for defining, altering and deleting database structures such as tables, indexes and schemas.
- It simply deals with descriptions of the database schema and is used to create and modify the structure of database objects in the database.

➢ **CREATE: -**

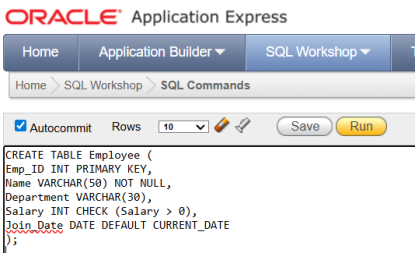
- Create database or its objects (table, index, function, views, store procedure and triggers).

- Syntax: CREATE TABLE table_name (column1 data_type, column2 data_type, ...);

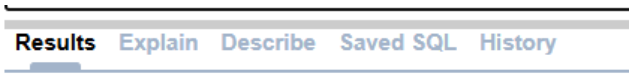
Step:

- **Run CREATE TABLE Employee (...)** → creates Employee table with constraints (PRIMARY KEY, NOT NULL, CHECK, DEFAULT).
- **Oracle creates the table successfully.**
- **Insert a record** into Employee table with ID, name, department, and salary.
- **Default JOIN_DATE is applied automatically.**
- **Run SELECT * FROM Employee;** → display stored record.
- Oracle shows **"Table created."**
- Output displays employee data with auto-filled join date.

Code:

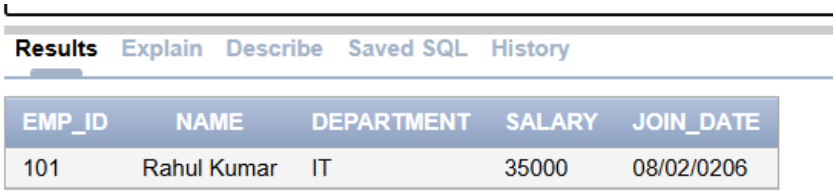


Output:




```
Table created.
```

0.07 seconds



EMP_ID	NAME	DEPARTMENT	SALARY	JOIN_DATE
101	Rahul Kumar	IT	35000	08/02/0206

1 rows returned in 0.01 seconds Download

➢ **DROP: -**

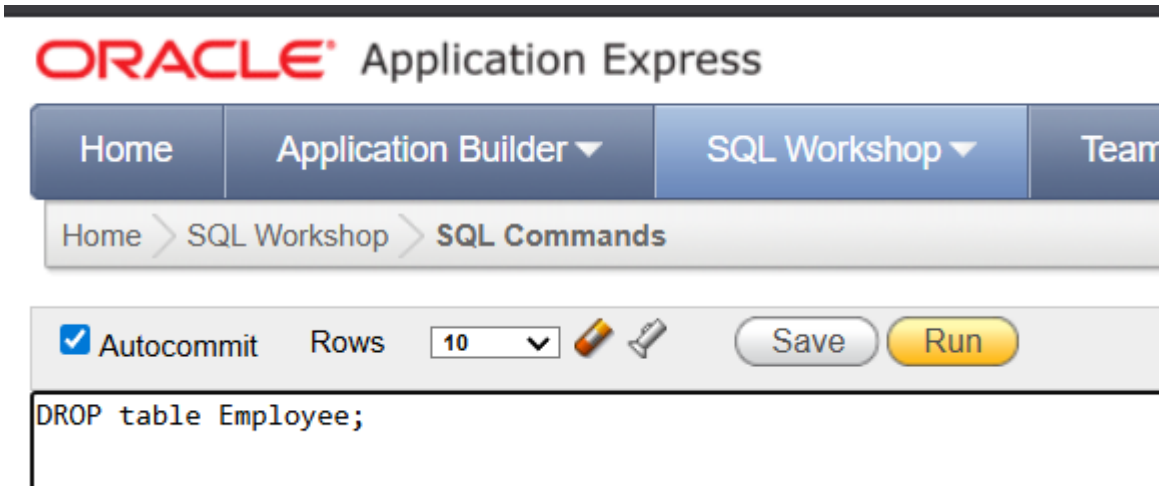
- Delete objects from the database.
- Syntax: DROP TABLE table_name
- DROP `TABLE Employee` permanently deletes the Employee table.
- Both **structure and data** are removed.
- This action **cannot be undone.**

Step:

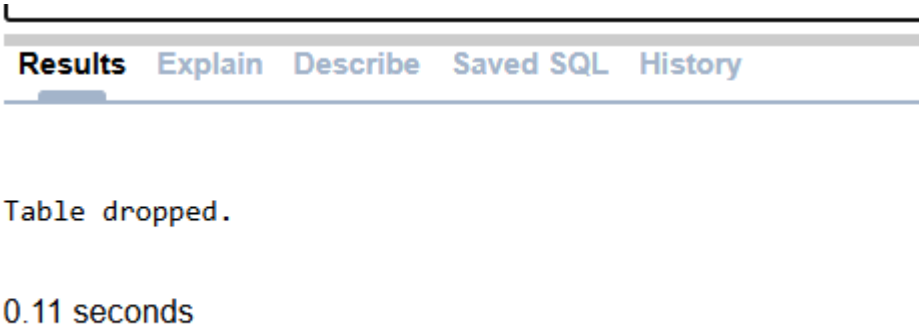
- **Write the command** DROP TABLE Employee; → to remove the Employee table.
- **Click Run** to execute the drop command.
- **Oracle processes the request** and deletes the table.
- **System confirms the action.**

- Oracle displays **"Table dropped."**
- The Employee table is permanently removed from the database.

Code:



Output:



➢ **ALTER: -**

- The ALTER TABLE statement is used to add, delete, or modify columns in an existing table.
- The ALTER TABLE statement is also used to add and drop various constraints on an existing table.

i.) ALTER TABLE - DROP COLUMN: -

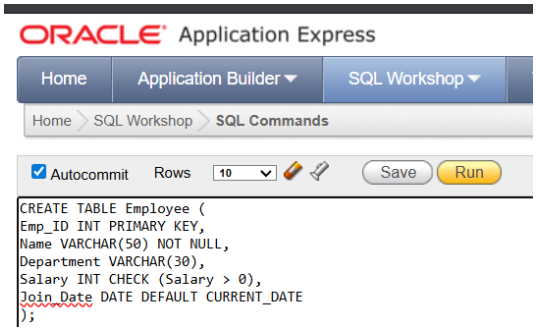
- DROP COLUMN Department → removes the Department column
- All data stored in that column is deleted
- Only the column is removed, the table still exists
- Syntax: ALTER TABLE *table_name* DROP COLUMN *column_name*;

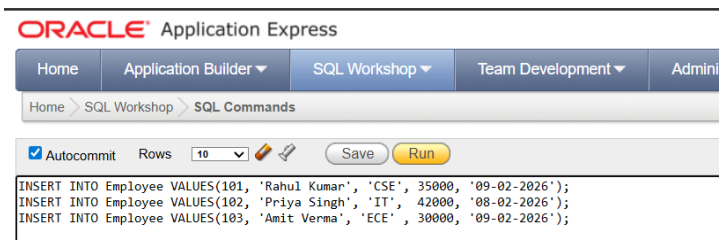
Step:

- **Write the command**
  ALTER TABLE Employee DROP COLUMN Department;
  → removes the Department column.
- **Click Run** to execute the alter command.
- **Oracle updates the table structure** and deletes that column.
- **Run SELECT * FROM Employee;** → view the modified table.
- Oracle displays **"Table altered."**
- Output shows the table without the Department column.

Code:

Before Alter





Results Explain Describe Saved SQL History

EMP_ID	NAME	DEPARTMENT	SALARY	JOIN_DATE
101	Rahul Kumar	CSE	35000	09/02/2026
102	Priya Singh	IT	42000	08/02/2026
103	Amit Verma	ECE	30000	09/02/2026

3 rows returned in 0.01 seconds Download

After Alter:

Code:

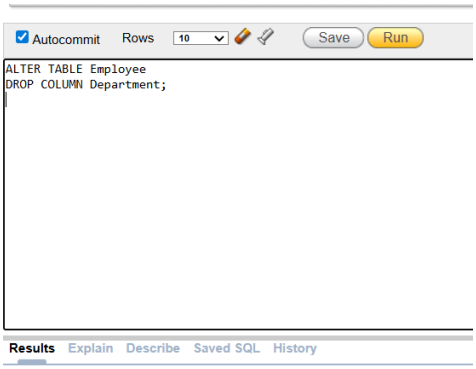


Table altered.

0.37 seconds

Output:

Results Explain Describe Saved SQL History

EMP_ID	NAME	SALARY	JOIN_DATE
101	Rahul Kumar	35000	09/02/2026
102	Priya Singh	42000	08/02/2026
103	Amit Verma	30000	09/02/2026

3 rows returned in 0.00 seconds Download

ii.) ALTER TABLE - RENAME COLUMN: -

- `ALTER TABLE Employee` → modifies the table
- Only the column name changes
- Data inside the column remains safe

Step:

- Run CREATE TABLE Student (...) → creates the Student table.
- Run INSERT commands → add records for Aarzoo, Priya, and Aman.
- Run SELECT * FROM Student; → display current table structure.
- Run ALTER TABLE Student RENAME COLUMN age TO Student_age; → rename column.
- Oracle displays "Table altered."
- Output shows column name changed to Student_age.

Before Alter:

```
SQL> create table Student (
  2  EMP_ID INT,
  3  NAME VARCHAR(50),
  4  AGE INT,
  5  DEPARTMENT VARCHAR(50)
  6  );

Table created.
```

```
EMP_ID NAME            AGE DEPARTMENT
------ --------------- ---- ----------
     1 Aarzoo           20 CSE
     3 Priya Singh      19 ECE
     4 Aman Verma       22 ME
```

```
SQL> INSERT INTO Student VALUES ( 1, 'Aarzoo',20,'CSE');

1 row created.

SQL> INSERT INTO Student VALUES (3, 'Priya Singh', 19, 'ECE');

1 row created.

SQL> INSERT INTO Student VALUES (4, 'Aman Verma', 22, 'ME');

1 row created.
```

After Alter:

```
SQL> ALTER table Student RENAME Column age TO Stusent_age;

Table altered.

SQL> select * from Student;

EMP_ID NAME            STUSENT_AGE DEPARTMENT
------ --------------- ----------- ----------
     1 Aarzoo                   20 CSE
     3 Priya Singh              19 ECE
     4 Aman Verma               22 ME

SQL>
```

iii.) ALTER TABLE - ADD Column: -

- The `ALTER TABLE ADD` command is used to add a new column to an existing table without deleting existing data.
- The new column becomes part of the table structure and can store values for future records.

Step:

- Run ALTER TABLE Student ADD Email VARCHAR(50); → adds a new Email column.
- Oracle modifies the table structure.
- Existing records remain unchanged.
- Run SELECT * FROM Student; → view updated table.
- Oracle displays "Table altered."
- Table shows the new Email column (empty for old records).

Code:

```
SQL> ALTER TABLE Student
  2  ADD Email VARCHAR(50);

Table altered.
```

Output:

```
EMP_ID NAME            STUDENT_AGE DEPARTMENT EMAIL
------ --------------- ----------- ---------- ------------
     1 Aarzoo                   20 CSE
     3 Priya Singh              19 ECE
     4 Aman Verma               22 ME

SQL>
```

iv.) ALTER TABLE - ALTER/MODIFY DATATYPE: -

- The `ALTER TABLE MODIFY` command is used to change the datatype or size of an existing column in a table.
- It allows updating the table structure without deleting the table or existing records.

Step:

- Run ALTER TABLE Student MODIFY Email VARCHAR(100); → increases email column size.
- Oracle alters the table structure successfully.
- Run INSERT command → adds Neha Sharma's record.
- Oracle stores the new row.
- Run SELECT * FROM Student; → view updated table.
- Oracle shows "Table altered" and "1 row created."
- Table displays the newly inserted record with email.

Code:

```
SQL> ALTER TABLE Student
  2  MODIFY Email VARCHAR(100);

Table altered.

SQL> INSERT INTO Student
  2  VALUES (5, 'Neha Sharma', 21, 'IT', 'neha@gmail.com');

1 row created.

SQL> select * from Student;

EMP_ID NAME            STUDENT_AGE DEPARTMENT EMAIL
------ --------------- ----------- ---------- --------------------
     1 Aarzoo                   20 CSE
     3 Priya Singh              19 ECE
     4 Aman Verma               22 ME
     5 Neha Sharma              21 IT         neha@gmail.com

SQL>
```

➢ **TRUNCATE: -**

- The `TRUNCATE` command removes **all records** from a table quickly but keeps the table structure intact.
- It is faster than DELETE and cannot be rolled back in most databases.

Step:

- Run SELECT * FROM Student; → view existing records in the table.
- Run TRUNCATE TABLE Student; → remove all rows instantly.
- Oracle clears the table data.
- Run SELECT * FROM Student; again → verify the table.
- Oracle displays "Table truncated."
- Output shows no rows selected (table is empty).

Before:

```
1 row created.

SQL> select * from Student;

EMP_ID NAME            STUDENT_AGE DEPARTMENT EMAIL
------ --------------- ----------- ---------- --------------------
     1 Aarzoo                   20 CSE
     3 Priya Singh              19 ECE
     4 Aman Verma               22 ME
     5 Neha Sharma              21 IT         neha@gmail.com
```

After:

```
SQL> TRUNCATE TABLE Student;

Table truncated.

SQL> select * from Student;

no rows selected

SQL>
```

ii.) DML (Data Manipulation Language): -

- DML commands are used to manipulate the data stored inside a database table.
- These commands allow inserting, updating, deleting, and retrieving records.
- DML operations do not change the structure of the table, only the data.
- The main DML commands are **INSERT, SELECT, UPDATE, and DELETE**.

➢ **INSERT Command: -**

- The INSERT command is used to add new records (tuples) into a table.
- It allows storing new data without affecting existing records.

Step:

- **Run first INSERT command** → adds record for Karan.
- **Run second INSERT command** → adds record for Simran.
- **Run third INSERT command** → adds record for Arjun.
- **Oracle stores all rows successfully.**
- **Run SELECT * FROM Student;** → display all records.
  - **Result:**
- Oracle shows **"1 row created"** for each insert.
- Table displays all three newly inserted records.

Code:

```
SQL> INSERT INTO Student
  2  VALUES (7, 'Karan', 22, 'ECE', 'karan@gmail.com');

1 row created.

SQL> INSERT INTO Student
  2  VALUES (8, 'Simran', 21, 'CSE', 'simran@gmail.com');

1 row created.

SQL> INSERT INTO Student
  2  VALUES (9, 'Arjun', 23, 'ME', 'arjun@gmail.com');

1 row created.
```

Output:

```
SQL> select * from Student;

    STD_ID NAME            STUDENT_AGE DEPARTMENT EMAIL
---------- --------------- ----------- ---------- --------------------
         7 Karan                    22 ECE        karan@gmail.com
         8 Simran                   21 CSE        simran@gmail.com
         9 Arjun                    23 ME         arjun@gmail.com

SQL>
```

- **SELECT Command: -**

  - The SELECT command is used to retrieve data from a database table.
  - It allows viewing all records or specific columns based on conditions.

```
SQL> select * from Student;

    STD_ID NAME            STUDENT_AGE DEPARTMENT EMAIL
---------- --------------- ----------- ---------- --------------------
         7 Karan                    22 ECE        karan@gmail.com
         8 Simran                   21 CSE        simran@gmail.com
         9 Arjun                    23 ME         arjun@gmail.com

SQL>
```

- **DELETE Command: -**

  - The DELETE command is used to remove specific records from a table.
  - It deletes data based on a condition without removing the table structure.

```
SQL> DELETE FROM Student WHERE Std_id=8;

1 row deleted.

SQL> select * from Student;

    STD_ID NAME            STUDENT_AGE DEPARTMENT EMAIL
---------- --------------- ----------- ---------- --------------------
         7 Karan                    22 ECE        karan@gmail.com
         9 Arjun                    23 ME         arjun@gmail.com

SQL>
```

- **UPDATE Command: -**

  - The UPDATE command is used to modify existing records in a table.
  - It changes data based on a specified condition without affecting other rows.

```
SQL> UPDATE Student
  2  SET DEPARTMENT = 'IT'
  3  WHERE Std_ID = 9;

1 row updated.

SQL> select * from Student;

    STD_ID NAME            STUDENT_AGE DEPARTMENT EMAIL
---------- --------------- ----------- ---------- --------------------
         7 Karan                    22 ECE        karan@gmail.com
         9 Arjun                    23 IT         arjun@gmail.com

SQL>
```

iii.) DCL – Data Control Language: -

- DCL commands are used to control access and permissions in a database.
- They help protect data by allowing only authorized users to perform operations.
- The main DCL commands are **GRANT** and **REVOKE**.
- DCL ensures database security and user privilege management.

➢ **GRANT Command:**

- The GRANT command is used to give specific permissions to a user on a database table.
- It controls who can access or modify the data.

```
SQL> GRANT SELECT, INSERT, UPDATE ON Student TO user1;
GRANT SELECT, INSERT, UPDATE ON Student TO user1
                                           *
ERROR at line 1:
ORA-01917: user or role 'USER1' does not exist
```

➢ **REVOKE Command: -**

- The REVOKE command is used to remove permissions previously granted to a user.
- It helps maintain database security by restricting access.

```
SQL> REVOKE INSERT, UPDATE ON Student FROM user1;
REVOKE INSERT, UPDATE ON Student FROM user1
                                         *
ERROR at line 1:
ORA-01917: user or role 'USER1' does not exist


SQL>
```

iv.) TCL – Transaction Control Language: -

- TCL commands are used to manage transactions in a database.
- They ensure data consistency by allowing changes to be saved or undone.
- The main TCL commands are **COMMIT, ROLLBACK, SAVEPOINT, and SET TRANSACTION**.
- TCL helps maintain data integrity during multiple database operations.

➢ **COMMIT Command:**

- The COMMIT command is used to permanently save all changes made in the current transaction.
- After commit, the changes cannot be undone using rollback.

Step:

- **Run UPDATE command →**
  UPDATE Student SET DEPARTMENT='CSE' WHERE STD_ID=7;
  modifies the record.
- **Run COMMIT;** → permanently saves the change.
- **Oracle confirms commit success.**
- **Run SELECT * FROM Student;** → verify updated data.
- Oracle shows **Commit complete**.
- The updated department value remains saved in the table.

```
SQL> UPDATE Student
  2  SET DEPARTMENT = 'CSE'
  3  WHERE STD_ID = 7;

1 row updated.

SQL>
SQL> COMMIT;

Commit complete.

SQL> select * from Student;

    STD_ID NAME            STUDENT_AGE DEPARTMENT EMAIL
---------- --------------- ----------- ---------- --------------------
         7 Karan                    22 CSE        karan@gmail.com
         9 Arjun                    23 IT         arjun@gmail.com

SQL>
```

- **ROLLBACK Command: -**
  - The ROLLBACK command is used to undo changes made in the current transaction.
  - It restores the database to the last committed state.

  Step:

  - Run UPDATE command →
    UPDATE Student SET DEPARTMENT='ME' WHERE STD_ID=9;
    record is temporarily modified.
  - Run ROLLBACK; → cancels the update.
  - Oracle restores previous data automatically.
  - Run SELECT * FROM Student; → verify table contents.
  - Oracle displays Rollback complete.
  - The department value returns to the original state (IT).

```
SQL> UPDATE Student
  2  SET DEPARTMENT = 'ME'
  3  WHERE STD_ID = 9;

1 row updated.

SQL>
SQL> ROLLBACK;

Rollback complete.

SQL> select * from Student;

    STD_ID NAME            STUDENT_AGE DEPARTMENT EMAIL
---------- --------------- ----------- ---------- --------------------
         7 Karan                    22 CSE        karan@gmail.com
         9 Arjun                    23 IT         arjun@gmail.com
```

- **SAVEPOINT Command: -**

- The SAVEPOINT command creates a temporary point inside a transaction to which you can roll back later.
- It allows partial undo of changes instead of cancelling the entire transaction.

Step:

- Run first UPDATE →
  UPDATE Student SET DEPARTMENT='CSE' WHERE STD_ID=7;
  record is modified.
- Create first savepoint →
  SAVEPOINT sp1;
  marks a recovery point.
- Run second UPDATE →
  UPDATE Student SET DEPARTMENT='ME' WHERE STD_ID=9;
- Create second savepoint →
  SAVEPOINT sp2;
- Run rollback →
  ROLLBACK TO sp2;
  restores table to state at savepoint sp2.
- Run SELECT * FROM Student; → check final data.

```
SQL> UPDATE Student
  2  SET DEPARTMENT = 'CSE'
  3  WHERE STD_ID = 7;

1 row updated.

SQL>
SQL> SAVEPOINT sp1;

Savepoint created.
```

```
SQL> UPDATE Student
  2  SET DEPARTMENT = 'ME'
  3  WHERE STD_ID = 9;

1 row updated.

SQL>
SQL> SAVEPOINT sp2;

Savepoint created.
```

```
SQL> ROLLBACK TO sp2;

Rollback complete.

SQL> select * from Student;

    STD_ID NAME            STUDENT_AGE DEPARTMENT EMAIL
---------- --------------- ----------- ---------- --------------------
         7 Karan                    22 CSE        karan@gmail.com
         9 Arjun                    23 ME         arjun@gmail.com

SQL>
```

- **SET TRANSACTION Command: -**
  - The SET TRANSACTION command is used to control the behavior of a transaction before it starts.
  - It defines properties like read-only or read-write mode for a transaction.

  Step:

  - **Run COMMIT;** → saves all previous changes permanently.
  - **Run SET TRANSACTION READ WRITE;** → starts a new editable transaction.
  - **Run the UPDATE command →**
    UPDATE Student SET DEPARTMENT = 'CSE' WHERE STD_ID = 9;
  - **Run SELECT * FROM Student;** → check the updated record.

```
SQL> COMMIT;

Commit complete.

SQL> SET TRANSACTION READ WRITE;

Transaction set.

SQL> UPDATE Student
  2  SET DEPARTMENT = 'CSE'
  3  WHERE STD_ID = 9;

1 row updated.

SQL> select * from Student;

    STD_ID NAME            STUDENT_AGE DEPARTMENT EMAIL
---------- --------------- ----------- ---------- --------------------
         7 Karan                    22 CSE        karan@gmail.com
         9 Arjun                    23 CSE        arjun@gmail.com

SQL>
```

4.) Implementation of I/O Constraint: Primary Key, composite primary key, Foreign Key with on delete set null and on delete set null constraint, Unique Key.

Objective: -To implement integrity constraints in a database including Primary Key, Composite Primary Key, Foreign Key with ON DELETE SET NULL, and Unique Key to maintain data accuracy and consistency.

- **Primary Key: -**
  - A primary key uniquely identifies each record in a table.
  - It does not allow duplicate or NULL values.
  - Each table can have only one primary key.
  - It ensures data integrity and prevents duplicate records.

  Step:
    - Run CREATE TABLE Student1 (...) → creates a table with STD_ID as PRIMARY KEY.
    - Run first INSERT command → adds record for Aarzoo successfully.
    - Run second INSERT command → adds record for Rahul successfully.
    - Run third INSERT command → fails because STD_ID = 1 is duplicate (primary key violation).
    - Run SELECT * FROM Student1; → displays only valid inserted records.
    - Oracle shows unique constraint violated error for duplicate ID.
    - Table contains only the first two valid rows.

```
SQL> CREATE TABLE Student1 (
  2  STD_ID INT PRIMARY KEY,
  3  Name VARCHAR(50),
  4  Age INT,
  5  Department VARCHAR(30),
  6  Email VARCHAR(50)
  7  );

Table created.
```

```
SQL> INSERT INTO Student1 VALUES  (1, 'Aarzoo', 20, 'CSE', 'aarzoo@gmail.com');

1 row created.

SQL> INSERT INTO Student1 VALUES   (2, 'Rahul', 21, 'IT', 'rahul@gmail.com');

1 row created.

SQL> INSERT INTO Student1 VALUES   (1, 'Priya', 19, 'ECE', 'priya@gmail.com');
INSERT INTO Student1 VALUES   (1, 'Priya', 19, 'ECE', 'priya@gmail.com')
*
ERROR at line 1:
ORA-00001: unique constraint (SYSTEM.SYS_C007101) violated
```

```
SQL> select * from Student1;

    STD_ID NAME            AGE DEPARTMENT EMAIL
---------- --------------- ---- ---------- --------------------
         1 Aarzoo           20 CSE        aarzoo@gmail.com
         2 Rahul            21 IT         rahul@gmail.com
```

- **Composite Primary Key: -**

  - A composite primary key is a key formed using **two or more columns** to uniquely identify a record.
  - A composite key ensures uniqueness using a combination of multiple attributes.
  - No two rows can have the same combination of those key values.

Step:

- Run CREATE TABLE Enrollment (...) → creates table with a composite primary key (Student_ID + Course_ID).
- Run first two INSERT commands → records (1,101) and (2,202) are added successfully.
- Run third INSERT command → fails because (1,101) is duplicate (composite key violation).
- Run another INSERT (1,102) → succeeds since the combination is unique.
- Run SELECT * FROM Enrollment; → displays all valid records.
- Oracle shows unique constraint violated for duplicate pair.
- Only unique Student_ID + Course_ID combinations remain in the table

```
SQL> CREATE TABLE Enrollment (
  2  Student_ID INT,
  3  Course_ID INT,
  4  PRIMARY KEY (Student_ID, Course_ID)
  5  );

Table created.
```

```
SQL> INSERT INTO Enrollment VALUES (1, 101);

1 row created.

SQL> INSERT INTO Enrollment VALUES (2, 202);

1 row created.

SQL> INSERT INTO Enrollment VALUES (1, 101);
INSERT INTO Enrollment VALUES (1, 101)
*
ERROR at line 1:
ORA-00001: unique constraint (SYSTEM.SYS_C007102) violated


SQL> INSERT INTO Enrollment VALUES (1, 102);

1 row created.

SQL>
```

```
SQL> select * from Enrollment;

STUDENT_ID  COURSE_ID
---------- ----------
         1        101
         2        202
         1        102

SQL>
```

- **Foreign Key with ON DELETE SET NULL: -**

  - A foreign key creates a link between two tables and maintains referential integrity.

  - The ON DELETE SET NULL constraint sets the foreign key value to NULL when the referenced parent row is deleted.

Step:

- Run CREATE TABLE Department (...) → creates Department table with primary key Dept_ID.
- Run CREATE TABLE Student2 (...) → creates Student table with a foreign key referencing Department.
- Insert department record using
  INSERT INTO Department VALUES (10, 'CSE');
- Insert student record using
  INSERT INTO Student2 VALUES (1, 'Aarzoo', 10);
- Delete department using
  DELETE FROM Department WHERE Dept_ID = 10;
  → foreign key rule ON DELETE SET NULL is triggered
- Department row is deleted successfully.
- In Student2 table, Dept_ID becomes NULL instead of deleting the student record.

```
Connected.
SQL> CREATE TABLE Department (
  2  Dept_ID INT PRIMARY KEY,
  3  Dept_Name VARCHAR(50)
  4  );

Table created.
```

```
SQL> CREATE TABLE Student2 (
  2  STD_ID INT PRIMARY KEY,
  3  Name VARCHAR(50),
  4  Dept_ID INT,
  5  FOREIGN KEY (Dept_ID)
  6  REFERENCES Department(Dept_ID)
  7  ON DELETE SET NULL
  8  );

Table created.

SQL>
```

```
SQL> INSERT INTO Department VALUES (10, 'CSE');

1 row created.

SQL>
SQL> INSERT INTO Student2 VALUES (1, 'Aarzoo', 10);

1 row created.

SQL> DELETE FROM Department WHERE Dept_ID = 10;

1 row deleted.
```

```
SQL> select * from Student2;

    STD_ID NAME                                               DEPT_ID
---------- -------------------------------------------------- ----------
         1 Aarzoo
```

- **Unique Key Constraint: -**

  - A unique key ensures that all values in a column are different from each other.
  - It prevents duplicate entries but allows one NULL value (in most DBMS).

  Step:

  - Run CREATE TABLE Teacher (...) → creates table with PRIMARY KEY and UNIQUE email constraint.
  - Insert first record → email teacher1@gmail.com is added successfully.
  - Insert second record with NULL email → allowed because UNIQUE allows NULL.
  - Insert duplicate email → fails due to UNIQUE constraint violation.
  - Run SELECT * FROM Teacher; → displays only valid records.
  - Oracle shows unique constraint violated for duplicate email.
  - Table contains only unique email values (and NULL allowed once or more).

```
SQL> CREATE TABLE Teacher (
  2  Teacher_ID INT PRIMARY KEY,
  3  Email VARCHAR(50) UNIQUE
  4  );
```

```
Table created.

SQL> INSERT INTO Teacher VALUES (1, 'teacher1@gmail.com');

1 row created.

SQL> INSERT INTO Teacher VALUES (2, NULL);

1 row created.

SQL> INSERT INTO Teacher VALUES (4, 'teacher1@gmail.com');
INSERT INTO Teacher VALUES (4, 'teacher1@gmail.com')
*
ERROR at line 1:
ORA-00001: unique constraint (SYSTEM.SYS_C007107) violated
```

```
SQL> select * from Teacher;

TEACHER_ID EMAIL
---------- --------------------------------
         1 teacher1@gmail.com
         2

SQL>
```

5.) Implementation of Business Constraint: Null, Not Null, Default, Check.

Objective: - To implement business constraints such as NULL, NOT NULL, DEFAULT, and CHECK to enforce rules and maintain data validity in a database.

- **NULL Constraint: -**

  - The NULL constraint allows a column to store empty or missing values.
  - It is used when a value is optional and not mandatory.

  Step:

  - Run CREATE TABLE Student3 (...) → creates table where Phone column allows NULL values.
  - Insert first record with NULL phone → allowed because column is optional.
  - Insert second record with a phone number → stored successfully.
  - Run SELECT * FROM Student3; → view the inserted records.
  - Oracle inserts both rows successfully.
  - Table shows one NULL phone value and one filled phone number.

```
SQL> CREATE TABLE Student3 (
  2  STD_ID INT PRIMARY KEY,
  3  Name VARCHAR(50),
  4  Phone VARCHAR(15) NULL
  5  );

Table created.
```

```
SQL> INSERT INTO Student3 VALUES (1, 'Aarzoo', NULL);

1 row created.

SQL> INSERT INTO Student3
  2  VALUES (2, 'Rahul', '9876543210');

1 row created.
```

```
SQL> select * from  Student3;

    STD_ID NAME                                               PHONE
---------- -------------------------------------------------- ---------------
         1 Aarzoo
         2 Rahul                                              9876543210

SQL>
```

- **NOT NULL Constraint: -**

  - The `NOT NULL` constraint ensures that a column must always contain a value.
  - It prevents insertion of empty or missing data in that column.

  Step:

  - Run CREATE TABLE Student4 (...) → creates table with Name column set as NOT NULL.
  - Insert first two records → valid names are inserted successfully.
  - Try inserting NULL in Name column → Oracle rejects the record.
  - Second NULL insert attempt also fails due to NOT NULL constraint.
  - Run SELECT * FROM Student4; → view only valid rows.
  - Oracle shows error "cannot insert NULL" for Name column.
  - Table contains only rows where Name is not NULL.

```
SQL> CREATE TABLE Student4 (
  2  STD_ID INT PRIMARY KEY,
  3  Name VARCHAR(50) NOT NULL,
  4  Phone VARCHAR(15)
  5  );

Table created.
```

```
SQL> INSERT INTO Student4 VALUES (1, 'Aarzoo', '9876543210');

1 row created.

SQL>
SQL> INSERT INTO Student4 VALUES (2, 'Rahul', '9123456780');

1 row created.

SQL> INSERT INTO Student4 VALUES (3, NULL, '9999999999');
INSERT INTO Student4 VALUES (3, NULL, '9999999999')
                             *
ERROR at line 1:
ORA-01400: cannot insert NULL into ("SYSTEM"."STUDENT4"."NAME")


SQL>
SQL> INSERT INTO Student4 VALUES (4, NULL, '8888888888');
INSERT INTO Student4 VALUES (4, NULL, '8888888888')
                             *
ERROR at line 1:
ORA-01400: cannot insert NULL into ("SYSTEM"."STUDENT4"."NAME")
```

```
SQL> select * from Student4;

    STD_ID NAME                                               PHONE
---------- -------------------------------------------------- ---------------
         1 Aarzoo                                             9876543210
         2 Rahul                                              9123456780

SQL>
```

- **DEFAULT Constraint: -**

  - The `DEFAULT` constraint automatically assigns a value to a column if no value is provided during insertion.
  - It prevents NULL values by giving a predefined value.

Step:

- Run CREATE TABLE Student5 (...) → creates table with Department column having DEFAULT 'CSE'.
- Insert records with department values → Rahul (IT) and Aman (ECE) stored normally.
- Insert record without department using column list → default value is applied.
- Insert another record without department → Oracle automatically fills CSE.
- Run SELECT * FROM Student5; → view all records.
- Records without department automatically get CSE.
- Table shows both manual and default department values.

```
SQL> CREATE TABLE Student5 (
  2  STD_ID INT PRIMARY KEY,
  3  Name VARCHAR(50),
  4  Department VARCHAR(30) DEFAULT 'CSE'
  5  );

Table created.
```

```
SQL> INSERT INTO Student5 VALUES (2, 'Rahul', 'IT');

1 row created.

SQL> INSERT INTO Student5 VALUES (4, 'Aman', 'ECE');

1 row created.

SQL> INSERT INTO Student5 (STD_ID, Name)
  2  VALUES (3, 'Priya');

1 row created.
```

```
SQL> INSERT INTO Student5 (STD_ID, Name)
  2  VALUES (1, 'Aarzoo');

1 row created.
```

```
SQL> select * from Student5;

STD_ID NAME         DEPARTMENT
------ ------------ ----------
     2 Rahul        IT
     4 Aman         ECE
     3 Priya        CSE
     1 Aarzoo       CSE

SQL>
```

- **CHECK Constraint: -**

  - The `CHECK` constraint restricts values in a column based on a condition.
  - It ensures only valid data is inserted into the table.

  Step:

  - Run CREATE TABLE Student6 (...) → creates table with a CHECK constraint on Age (18–30).
  - Insert first record with age 20 → valid and inserted successfully.
  - Insert second record with age 15 → violates CHECK constraint.
  - Oracle rejects the invalid record automatically.
  - Run SELECT * FROM Student6; → view only valid data.
  - Oracle shows check constraint violated error.
  - Table contains only rows where age is between 18 and 30.

```
SQL> CREATE TABLE Student6 (
  2  STD_ID INT PRIMARY KEY,
  3  Name VARCHAR(50),
  4  Age INT CHECK (Age >= 18 AND Age <= 30)
  5  );

Table created.
```

```
SQL> INSERT INTO Student6 VALUES (1, 'Aarzoo', 20);

1 row created.

SQL> INSERT INTO Student6 VALUES (2, 'Rahul', 15);
INSERT INTO Student6 VALUES (2, 'Rahul', 15)
*
ERROR at line 1:
ORA-02290: check constraint (SYSTEM.SYS_C007115) violated
```

```
SQL> select * from Student6;

STD_ID NAME                 AGE
------ ------------ ----------
     1 Aarzoo                20

SQL>
```